

The Splitsy logo, featuring a stylized 'S' with a small icon of a bill or receipt inside the top curve, followed by the word 'plitsy' in a sans-serif font.

# Splitsy



**Split any bill on-chain. Let an agent do the work.**

Programmable Money Hackathon



**Agentic Economy track ·  DeFi track**

[splitsy.xyz](https://splitsy.xyz) · [github.com/qFloppa/Splitsy](https://github.com/qFloppa/Splitsy)

# The problem

Splitting a bill still ends in a group chat full of IOUs. "I'll send it later," and then nobody does.

-  Crypto-native splitting exists, but it assumes **everyone already holds a wallet and a gas token.**
-  Debts live in someone's notes app, so there is **no shared record and no consequence** for never paying.
-  The receipt work itself, scanning and parsing and currency conversion, is **labour nobody accounts for.**

“Splitsy makes the debt real, makes joining frictionless, and makes the scanning pay for itself.”

# What Splitsy does

**Snap a receipt, split it, everyone pays their share in USDC on Arc.**

1.  **Scan.** AI parses merchant, line items, tax, tip, and currency.
2.  **Split.** Assign shares, including to people with no wallet yet.
3.  **Escrow.** Each debt is committed on-chain to the `BillSplitRegistry` contract.
4.  **Settle.** Payers pay real USDC, the creator claims it, reputation is written on-chain.

Splitsy never takes custody. The contract holds the funds and the truth.



# Agentic Economy

Scout goes shopping



# Upload a receipt. An agent goes shopping.

**Scout** is an autonomous agent with a real economy of its own.

- Its own **wallet and ERC-8004 identity** on Arc.
- A **daily budget of \$0.05** that it will not exceed.
- It **judges your photo before spending anything**.
- It **pays Splitsy's own x402 endpoints per call** in USDC fractions.
- It **buys a second opinion** when the parse looks shaky.

“Splitsy is both the buyer's principal and the seller. It is a closed loop you can watch settle live on chain.”



# x402 nanopayments on Arc

| Endpoint              | Price          | Service                    |
|-----------------------|----------------|----------------------------|
| <code>/api/ocr</code> | <b>\$0.005</b> | Parse a receipt            |
| <code>/api/fx</code>  | <b>\$0.001</b> | Convert a foreign currency |

1. Scout POSTs. The seller answers `402 Payment Required` with a base64 `PAYMENT-REQUIRED` challenge naming scheme, network, asset, and amount.
2. Scout signs a **gasless EIP-3009 authorization**. No transaction, no gas.
3. The facilitator runs `verify()` then `settle()`, and **Circle Gateway batches** it on Arc.
4. `200 OK` with a `PAYMENT-RESPONSE` transaction hash, ledgered as **earned**.



# Scout's decision logic

Pure functions, fully unit-tested, because this is the logic that spends money.

- 🔍 **Assess first.** Reject images too small or too low-resolution **before paying anything.**
- 💰 **Budget gate.** Spend checks run in atomic base units, never floats, so the cap holds exactly at the boundary.
- 🔄 **Second opinion.** If confidence is **below 0.80** *and* the budget allows, pay again for a high-rigour re-scan and keep the better parse.
- 🌐 **Foreign currency.** Pay the FX seller only when the bill is not already in USD.
- 🚫 **Graceful degradation.** If the paid path fails, fall back to a direct parse. **The human upload never breaks.**



# One scan, one live agent economy

## 3 nanopayments · \$0.011

| Call                                         | Paid    | Result                             |
|----------------------------------------------|---------|------------------------------------|
| <code>/api/ocr</code>                        | \$0.005 | confidence 0.62, below the gate ⚠️ |
| <code>/api/ocr</code> <code>{hq:true}</code> | \$0.005 | confidence 0.94, kept ✅            |
| <code>/api/fx</code>                         | \$0.001 | EUR is not USD 🌐                   |

The dashboard's agent-economy panel tracks **earned against spent**, calls served against calls paid, and **budget remaining**, updating as Scout works.





# DeFi

**The net-settlement treasury**

 **Many debts. One position.**

Real groups owe each other across dozens of bills. The dashboard collapses all of it into **one net figure per counterparty**, sorted by largest exposure.

| Counterparty    | Owed to me | I owe   | Net      |
|-----------------|------------|---------|----------|
| @dev            | \$0.00     | \$43.75 | -\$43.75 |
| @carla          | \$18.40    | \$0.00  | +\$18.40 |
| sam@example.com | \$7.00     | \$0.00  | +\$7.00  |

Counterparties resolve to live handles rather than raw hex. Both directions net into a single row per person.

## ⚡ Settle net in one atomic transaction

Settling bill by bill costs an approve plus a `payDebt` per debt, plus one `claim` each. Five debts and four claims is  $2 \times 5 + 4$ .

### 14 transactions → 1

- **Circle SCA wallet:** one `executeBatch((target,value,data)[[]])` on the wallet's own address bundles every approval, payment, and claim into **one atomic transaction**. All of it lands or none does.
- 🦊 **Browser EOA:** one approval covers every payment, then one transaction per leg.

“Escrow binds each debt to its `billId`, so batching removes **transactions, not transfers**. The net figure is exposure. We do not claim less USDC moves.”

# Why the treasury is real infrastructure

-  **Pure aggregation core.** Registry reads in, net positions out. All money math in base-unit `bigint`, one formatter at the wire boundary, and `bigint` never crosses `Response.json`.
-  **Chain-verified execution.** Both settle paths **re-read every outstanding amount from chain before signing**. A stale dashboard can never sign a wrong amount, and neither path accepts a client-supplied figure.
-  **Atomic batching** via Circle's `contractExecution` endpoint on `ARC-TESTNET`.
-  **Tested.** The aggregation core and the `executeBatch` encoder both carry unit tests, including a round-trip decode of every leg in order.

## Anyone can be in a split

Sign in with **X, Discord, Google, or email**, and Splitsy provisions a Circle smart-contract wallet on Arc behind the login. No seed phrase, no extension.

-  Tag someone with **no account at all**. Their share sits in escrow on Arc until they claim it with a handle, an inbox, or an address.
-  One bill can mix **social identities and non-custodial wallets** in the same member list. An X handle, a Discord username, an email address, and a raw `0x` address can all owe on the same receipt.
-  The creator picks which identity **creates** the bill too: the Circle wallet or a connected browser wallet.

Recurring tabs accept the same mixed membership, so a group never has to standardise on one wallet type.



# Bring USDC from another chain

Someone's USDC is rarely on the chain you need. Splitsy pulls it in with **Circle's CCTP v2**, native burn-and-mint, through App Kit.

## 6 source chains

Base Sepolia · Ethereum Sepolia · Arbitrum Sepolia · Optimism Sepolia ·  
Avalanche Fuji · Polygon Amoy

approve → burn → attest → mint

-  **No wrapped assets and no third-party bridge.** Native USDC out, native USDC in.
-  A **paymaster** path covers source-chain gas when the wallet's native balance is too low to bridge.

## Verifiable, escrowed debt

Every bill commits a **metadata hash** on-chain at creation.

-  Before paying, the payer's client **recomputes that hash from the off-chain preimage**. Merchant, total, and their own share must match what Arc records, or the UI warns them not to pay.
-  `payDebt` escrows USDC **per** `billId`. The creator calls `claim` to collect.
-  A **Circle Smart Contract Platform event monitor** watches the registry's `DebtPaid` event, so payments sent straight from a browser wallet, which never touch Splitsy's servers, still fire webhooks and still earn reputation.

Trust-minimised by construction: the app cannot move, redirect, or withhold anyone's money.



## "Pay by": a deadline, committed up front

The creator can set an **optional** due date. Leaving it blank keeps the bill exactly as it would be if due dates did not exist.

The date is **hashed into the bill's on-chain metadata at creation**, which is the whole point.

- 🚫 The creator **cannot move it, add one, or backdate it** afterwards. Doing so breaks the hash the payer verifies.
- 👁️ Payers see the deadline **before** they pay, framed as what it is: paying on time keeps your reputation strong.

A committed deadline is a promise the creator makes too, not a lever they hold.



# Why "Pay by" cannot grief anyone

## Worst case: 50 / 100

| Situation                     | Score                               | Tag          |
|-------------------------------|-------------------------------------|--------------|
| No deadline set               | 100                                 | paid_in_full |
| Within due date + 2-day grace | 100                                 | paid_on_time |
| Past grace                    | 100 less 5 per day, <b>floor 50</b> | paid_late    |

- **+ Paying is always positive.** Even very late, a completed payment is good faith, so the floor is a passing 50. No zero, no negative.
-  A **two-day grace window** absorbs timezone slack and "paid the morning it was due".
- **1** Scoring fires only when a payer **settles their full remaining share**, and each payment is scored exactly once.

## ERC-8004 payment reputation

Portable, verifiable reputation on the ERC-8004 registries Arc pre-deploys.

-  **The payer owns their identity NFT.** The registrar mints it, then transfers it to the payer.
-  ERC-8004 forbids scoring your own agent, so **the registrar scores, never the payer.** A `(wallet, bill_id)` constraint makes double-minting and double-scoring impossible.
-  Each score commits `keccak256("splitsy:bill:<id>:<payTx>")`, so **anyone can recompute a score against the exact payment it grades.**
-  Timing is graded on the `payDebt` **block timestamp**, not a server clock, identically across the Circle wallet, webhook, and replay paths.
-  The badge aggregate is **amount-weighted by share**, so a large late bill matters more than a small one.

## Recurring tabs: subscriptions on Arc

For weekly shared bills and monthly services, Splitsy deploys a **recurring tab contract** with a recipient, a cycle length, member wallets, and fixed USDC shares.

### **Allowance-based, not prepaid.**

1. 🖊️ Each payer approves the tab contract **once**, for a USDC limit they choose.
2. 🏛️ **Funds stay in the payers' own wallets** until a cycle is actually due.
3. 🕒 A backend settler calls `settleTab()` hourly. The contract pulls the fixed share from every member with enough balance and allowance, **skips the rest**, and makes the collected USDC claimable by the recipient.
4. 🔓 A payer **revokes at any time** by setting the tab allowance back to `0`.

Each cycle is scored independently for reputation, graded against the cycle boundary.

# The stack, with receipts

| Layer                      | What it is                                                        |
|----------------------------|-------------------------------------------------------------------|
| <b>Arc Testnet</b>         | chainId <code>5042002</code> , sub-second finality, Malachite BFT |
| <b>USDC-native gas</b>     | <code>0x3600...0000</code> . A share and its fee are one asset    |
| <b>Circle Wallets</b>      | accountType: "SCA" on <code>ARC-TESTNET</code> , per login        |
| <b>ERC-4337 + ERC-1967</b> | proxy over <code>circle_6900_singleowner_v3</code>                |
| <b>CCTP v2</b>             | native burn-and-mint from 6 testnet chains                        |
| <b>x402</b>                | <code>@circle-fin/x402-batching</code> via Circle Gateway         |
| <b>ERC-8004</b>            | IdentityRegistry <code>0x8004A818...</code>                       |
| <b>Circle SCP</b>          | <code>DebtPaid</code> event monitor to webhooks                   |

Next.js 16 · React 19 · viem 2.52 · Supabase · Multicall3 batched reads



## Why this fits both tracks

⚡ **Agentic Economy.** A genuine closed-loop agent economy: an autonomous agent with an **on-chain identity, a budget, and its own judgement**, paying **per-call x402 nanopayments** for services it decides it needs, while Splitsy earns as the seller on the other side.

🏛️ **DeFi.** A working treasury over escrowed on-chain debt: cross-bill net-position aggregation, then **atomic batched settlement**, every amount re-read from chain before signing, and no custody anywhere in the path.

“One product. Real USDC on Arc. An agent that pays its own way.”



Splitsy



**Thank you**  
**splitsy.xyz**

[github.com/qFloppa/Splitsy](https://github.com/qFloppa/Splitsy)

Arc · Circle Wallets · CCTP v2 · x402 · ERC-8004

Programmable Money Hackathon